

Breadth First Search & Iterative Depth First Search

AIM: Breadth First Search & Iterative Depth First Search

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai

sheth I u jhaveri college of arts and sir m v college of science and commerce

Practical no. 1

Breadth First Search & Iterative Depth First Search

NAME :Name: Adarsh Shukla
Roll no :no.: t117
Sub: ai

Breadth First Search & Iterative Depth First Search

```
from collections import deque
import time

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': ['H'],
    'F': [],
    'G': [],
    'H': []
}

def bfs(graph, start, goal):
    queue = deque([start])
    visited = set()

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")

            if node == goal:
                return True

            visited.add(node)

            for neighbour in graph[node]:
                if neighbour not in visited:
                    queue.append(neighbour)

    return False

def iterative_dfs(graph, start, goal):
    stack = [start]
    visited = set()

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")

            if node == goal:
                return True

            visited.add(node)

            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(neighbour)

    return False

start_node = 'A'
goal_node = 'H'

print("Breadth First Search:")
start_time = time.perf_counter()

bfs_result = bfs(graph, start_node, goal_node)
```

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai

Breadth First Search & Iterative Depth First Search

```
        return result

    return None

def iddfs(graph, start, goal, max_depth):
    print("\nIDDFS Iterations:")

    for depth in range(max_depth + 1):
        print("Searching with depth limit:", depth)

        result = depth_limited_search(
            graph,
            start,
            goal,
            depth,
            [start]
        )

        if result is not None:
            return result

    return None

print("Hariprasad Vishwakarma T127")
print()

bfs_path = bfs(graph, "Start", "Goal")
print("\nBFS Result Path:", bfs_path)

iddfs_path = iddfs(graph, "Start", "Goal", 10)
print("\nIDDFS Result Path:", iddfs_path)

goal_node = 'H'

print("Breadth First Search:")
start_time = time.perf_counter()

bfs_result = bfs(graph, start_node, goal_node)

bfs_time = time.perf_counter() - start_time

print("\nGoal Found:", bfs_result)
print("Execution Time:", bfs_time)

print("\n-----")

print("Iterative Depth First Search:")
start_time = time.perf_counter()

dfs_result = iterative_dfs(graph, start_node, goal_node)

dfs_time = time.perf_counter() - start_time

print("\nGoal Found:", dfs_result)
print("Execution Time:", dfs_time)

print("\n-----")

print("Performance Comparison")

if bfs_time < dfs_time:
    print("BFS executed faster.")
elif dfs_time < bfs_time:
    print("Iterative DFS executed faster.")
```

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai

Breadth First Search & Iterative Depth First Search

```
"ai p1t117.py - C:\Users\Adarsh\OneDrive\Videos\ai p1t117.py (3.12.6)"
File Edit Format Run Options Window Help

graph = {
    "Start": ["N1", "N2"],
    "N1": ["DeadEnd1", "DeadEnd2"],
    "N2": ["N3"],
    "N3": ["N4", "DeadEnd3"],
    "N4": ["Goal"],
    "DeadEnd1": [],
    "DeadEnd2": [],
    "DeadEnd3": [],
    "Goal": []
}

def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = set()

    print("BFS Exploration Order:")

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node in visited:
            continue

        print("Exploring node:", node)
        visited.add(node)

        if node == goal:
            return path

    return None

def depth_limited_search(graph, node, goal, limit, path):
    if node == goal:
        return path

    if limit == 0:
        return None

    for neighbour in graph[node]:
        if neighbour not in path:
            result = depth_limited_search(
                graph,
                neighbour,
                goal,
                limit - 1,
                path + [neighbour]
            )

            if result is not None:
                return result

    return None

def iddfs(graph, start, goal, max_depth):
    print("\nIDDFS Iterations:")

    for depth in range(max_depth + 1):
        print("Searching with depth limit:", depth)

        result = depth_limited_search(
```

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai

Practical no. 1

Breadth First Search & Iterative Depth First Search

```

    return result

    return None

def iddfs(graph, start, goal, max_depth):
    print("\nIDDFS Iterations:")

    for depth in range(max_depth + 1):
        print("Searching with depth limit:", depth)

        result = depth_limited_search(
            graph,
            start,
            goal,
            depth,
            [start]
        )

        if result is not None:
            return result

    return None

print("Hariprasad Vishwakarma T127")
print()

bfs_path = bfs(graph, "Start", "Goal")
print("\nBFS Result Path:", bfs_path)

iddfs_path = iddfs(graph, "Start", "Goal", 10)
print("\nIDDFS Result Path:", iddfs_path)

```

Output :

```

Python 3.12.6 (tags/v3.12.6:a4a2d2b, Sep  6 2024, 20:11:23) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Adarsh\OneDrive\Videos\ai p1T117.py
Breadth First Search:
A B C D E F G H
Goal Found: True
Execution Time: 0.011771699999826524

-----
Iterative Depth First Search:
A B D E H
Goal Found: True
Execution Time: 0.0070691000000806525

-----
Performance Comparison
Iterative DFS executed faster.
>>>

```

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai

Practical no. 1

Breadth First Search & Iterative Depth First Search

```
>>>
===== RESTART: C:\Users\Adarsh\OneDrive\Videos\ai p2 T117.py =====
Hariprasad Vishwakarma T127

BFS Exploration Order:
Exploring node: Start
Exploring node: N1
Exploring node: N2
Exploring node: DeadEnd1
Exploring node: DeadEnd2
Exploring node: N3
Exploring node: N4
Exploring node: DeadEnd3
Exploring node: Goal

BFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']

IDDFS Iterations:
Searching with depth limit: 0
Searching with depth limit: 1
Searching with depth limit: 2
Searching with depth limit: 3
Searching with depth limit: 4

IDDFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']
>>>

Goal Found: True
Execution Time: 0.0070691000000806525

-----
Performance Comparison
Iterative DFS executed faster.
>>>
===== RESTART: C:\Users\Adarsh\OneDrive\Videos\ai p2 T117.py =====
Hariprasad Vishwakarma T127

BFS Exploration Order:
Exploring node: Start
Exploring node: N1
Exploring node: N2
Exploring node: DeadEnd1
Exploring node: DeadEnd2
Exploring node: N3
Exploring node: N4
Exploring node: DeadEnd3
Exploring node: Goal

BFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']

IDDFS Iterations:
Searching with depth limit: 0
Searching with depth limit: 1
Searching with depth limit: 2
Searching with depth limit: 3
Searching with depth limit: 4

IDDFS Result Path: ['Start', 'N2', 'N3', 'N4', 'Goal']
>>>
```

NAME :Name: Adarsh Shukla

Roll no :no.: t117

Sub: ai